Efficient Planning Using Prioritized Sweeping for Autonomous Robot Navigation

CHAPTER 1: INTRODUCTION

1.1 Autonomous Robot Navigation

Autonomous robots are used in warehouses, manufacturing units, hospitals, laboratories and service environments. To perform tasks independently, a robot needs to determine where it is, where it must go and which movements can take it to the target safely. Navigation planning is one part of this process. It converts an environment representation into a sequence of movement decisions.

A navigation problem can be represented as a graph. Locations are represented by nodes and legal movements are represented by edges. In a grid-world abstraction, each free grid cell is a state. A route is an ordered sequence of states connected by valid actions. The robot's start position and goal position define the beginning and destination of the route.

A useful route should reach the goal, avoid obstacles and use a reasonable travel cost. Depending on the application, other considerations may include turning effort, energy use, risk and computation time. This project focuses on the planning aspect using a manageable software simulation.

1.2 Challenges in Dynamic Environments

A static map assumes that obstacles and free spaces remain unchanged. In a dynamic setting, an obstacle may appear, disappear or move. A path that was valid earlier can then pass through a blocked location. Continuing to follow the old path without checking it may cause failure.

The system therefore needs to validate a planned route and respond to changes. One approach is to run a complete path search again. A planning approach that maintains values and a transition model can instead update information influenced by the changed region. Prioritized Sweeping is relevant because it schedules value backups according to their estimated importance.

- Map changes can invalidate previously selected actions.
- Planning must recognize when the goal becomes unreachable.
- The model must be updated consistently after obstacles change.
- Evaluation should distinguish path quality from computational effort.

1.3 Reinforcement Learning and Planning

Reinforcement Learning (RL) studies how an agent learns decision-making behavior through interaction with an environment. The main elements include states, actions, rewards, transitions and a policy. A reward provides feedback about the desirability of an outcome, while a value function estimates expected future return.

Model-based planning uses a model of how actions change the state and what rewards they produce. The model can be used to perform simulated updates without requiring a new physical

movement for every update. Prioritized Sweeping is a planning technique that focuses computation on updates expected to have a greater effect on value estimates.

In navigation, a state can correspond to a free cell and an action can represent moving up, down, left or right. A terminal goal state ends an episode. The precise reward convention, transition behavior and terminal-state handling must be defined by the implementation.

1.4 Scope and Report Organization

The project scope is a software-based study of navigation planning in a grid environment. It covers environment representation, planning logic, route extraction, route validation and a planned response to map changes. The work is intended to explain and evaluate the algorithmic planning concept.

This report does not claim physical robot deployment, real sensor-based obstacle detection, localization, motor control or safety certification. These would require additional hardware or robotics simulation integration.

Chapter 2 reviews relevant planning concepts. Chapter 3 defines the problem and objectives. Chapter 4 presents system requirements. Chapter 5 explains the design and algorithm. Chapter 6 describes implementation modules. Chapters 7 and 8 provide the evaluation plan and verification matrix. Chapter 9 summarizes conclusions and future directions.

CHAPTER 2: LITERATURE REVIEW

2.1 Classical Path Planning Algorithms

Classical path-planning methods provide important reference points for autonomous navigation. Breadth-First Search (BFS) explores an unweighted graph level by level. When every movement has equal cost, BFS can find a path with the minimum number of edges if a route exists. Its behavior is straightforward and makes it a useful baseline for small grid maps.

Dijkstra's algorithm supports non-negative movement costs and finds minimum-cost paths by expanding states according to their accumulated cost. A* combines the accumulated path cost with a heuristic estimate of the remaining cost. For a four-direction grid with uniform movement costs, Manhattan distance is a common heuristic.

These search algorithms compute routes through graph exploration. Prioritized Sweeping differs in its central operation: it performs value-function backups using a model and prioritizes which states to update. Thus, the number of search expansions and the number of value backups are different quantities and should not be treated as directly equivalent.

2.2 Reinforcement Learning in Navigation

Reinforcement learning can formulate navigation as sequential decision-making. The agent receives a state description, chooses an action and observes a transition and reward. The objective is generally to maximize expected cumulative reward. Reward design can encourage reaching the goal and discourage collisions or unnecessarily long routes.

Value-based reinforcement learning methods estimate how beneficial states or state-action pairs are over time. In a deterministic, small grid, a tabular representation may be sufficient for demonstration. Larger or continuous environments may require function approximation or other representations.

The use of RL does not automatically ensure safe or optimal navigation. Results depend on the environment model, reward design, training process and evaluation coverage. For this reason, route validation and baseline comparisons remain necessary.

2.3 Model-Based Planning

A model-based approach uses a representation of state transitions and rewards to reason about possible outcomes. In a known grid-world, the transition model can be constructed from legal movements and obstacle occupancy. When an action is invalid, the implementation must define whether the agent remains in place, receives a penalty, or treats the action as unavailable.

Planning with a model can reuse computed information. However, if the model is inaccurate or is not updated when the map changes, planning can produce unsuitable actions. Maintaining transition and predecessor relationships is therefore important when implementing a predecessor-based update strategy.

2.4 Prioritized Sweeping

Prioritized Sweeping was introduced as a reinforcement-learning method intended to direct updates toward states where changes are expected to matter. Instead of repeatedly updating all states in a fixed order, it uses a priority mechanism. A common priority is the absolute difference between the current state value and a Bellman backup target.

A state with a large discrepancy may be placed in a priority queue. The planner removes a high-priority state, updates its value, and then examines predecessor states that can lead to it. If a predecessor's backup discrepancy is sufficiently large, that predecessor can also be queued. The process continues until the queue is empty or a chosen planning budget is reached.

The technique's efficiency is context-dependent. Queue management, model size, update thresholds, graph structure and implementation details affect its performance. It is not correct to claim that it always requires less time than BFS or A*.

2.5 Research Motivation

The project provides an educational setting for studying how prioritized value updates can support navigation decisions. A compact grid-world allows the relationship between a map, state transitions, predecessor links and planning priorities to be visualized.

The practical motivation is to understand whether the planning mechanism responds correctly to blocked routes and whether its operation can be measured against a clearly defined baseline. A fair study should use the same maps, movement rules, start and goal conditions, and repeated evaluation where appropriate.

CHAPTER 3: PROBLEM STATEMENT & OBJECTIVES

3.1 Problem Statement

An autonomous robot must travel from a start location to a goal in an environment containing obstacles. When the environment changes, a previously computed route may no longer be valid. The navigation system must identify invalid routes, update its planning information and produce a feasible alternative when one exists.

The problem addressed by this project is to study a grid-based navigation planning workflow using Prioritized Sweeping. The system represents free locations as states and movement choices as actions. It uses a transition and reward model to update value estimates, derives a route from the resulting policy, and validates that route against the current map.

3.2 Navigation Challenges

1. Obstacle avoidance: the route must not pass through blocked cells.
2. Dynamic changes: a newly blocked cell may require a new route.
3. Reachability: the planner must report when no feasible route exists.
4. Model correctness: transitions and predecessor relationships must match the current map.
5. Planning effort: updates and elapsed time should be recorded clearly.
6. Route extraction: a policy can cycle, so extraction needs a step bound or visited-state protection.

3.3 Aim and Objectives

Aim: To design and evaluate a software-based autonomous navigation planner that demonstrates the Prioritized Sweeping approach in a grid-world environment.

- Represent the navigation map as states, actions and transitions.
- Define a reward or cost convention for movement and goal completion.
- Implement prioritized value updates using a priority queue, if present in the final program.
- Use predecessor information to propagate relevant updates.
- Extract a route and verify each move against the map.
- Demonstrate the response to a map update and re-planning.
- Evaluate route validity and path cost, along with clearly defined planning-effort measures.
- Document test cases, limitations and reproducible settings.

CHAPTER 4: SYSTEM ANALYSIS AND REQUIREMENTS

4.1 System Overview and User Requirements

The proposed application is an educational simulation intended for students and evaluators. The user configures a grid, start and goal cells, and obstacle locations. The planning module processes the environment and produces a route or a clear no-route outcome. In a dynamic test, the map is modified and the route is checked or planned again.

The expected user needs are simple configuration, understandable visual output, reproducible experiments and transparent reporting of failure cases. The implementation should separate the environment representation from the planning algorithm so that each component can be verified independently.

4.2 Functional Requirements Specification

ID	Requirement
FR-01	Allow a grid map and obstacle configuration to be defined.
FR-02	Accept start and goal locations and validate coordinates.
FR-03	Represent free cells, legal actions and successor states.
FR-04	Construct or update transition and predecessor information.
FR-05	Execute the implemented planning algorithm.
FR-06	Extract a route or report that no route is available.
FR-07	Validate route continuity and obstacle avoidance.
FR-08	Support a map-change scenario and route revalidation/re-planning, if implemented.
FR-09	Record planning and route metrics supported by the program.
FR-10	Display the map, route and result in a comprehensible form.

4.3 Non-Functional Requirements Specification

- Usability: map and result visualizations should be easy to interpret.
- Reliability: invalid coordinates and unreachable goals should be handled safely.
- Reproducibility: fixed maps and settings should lead to consistent outcomes for deterministic implementations.
- Maintainability: environment, model, planner and evaluation logic should be organized clearly.

- Performance: the application should run within the available notebook or computer resources.
- Traceability: recorded figures and tables must correspond to identifiable program runs.

4.4 Hardware and Software Requirements

Component	**Suggested/expected environment**	**Purpose**
Programming language	Python 3.x (confirm exact version)	Algorithm and simulation development
Notebook environment	Google Colab or Jupyter (confirm actual use)	Run and document experiments
Numerical package	NumPy, if used	Array and numerical operations
Visualization	Matplotlib, if used	Map and route display
Hardware	Laptop/desktop or hosted notebook runtime	Execute the simulation
Input	Grid definition / obstacle map	Describe navigation environment

CHAPTER 5: METHODOLOGY / SYSTEM DESIGN

5.1 Formal Markov Decision Process (MDP) Formulation

A Markov Decision Process can be represented as (S, A, P, R, γ), where S is the state set, A is the action set, P describes transition probabilities, R describes rewards, and γ is the discount factor. In a deterministic grid-world, each legal action may lead to one known successor state.

State: a traversable cell, represented by its row and column coordinates. The current location is one state; a path is a sequence of states. This distinction matters when interpreting the route output.

Actions: a common action set contains movement up, down, left and right. Diagonal movement should be included only if the program explicitly supports it. Moves outside the grid or into obstacles must follow a defined rule.

Transition: the model identifies the successor state produced by an action. Reward: a possible convention is a step cost for movement and a positive terminal reward for reaching the goal. The final report must use the reward values and terminal handling in the actual implementation.

Discount factor: γ controls the relative contribution of future returns. For finite episodes, terminal-state values and the update behavior should be stated consistently.

5.2 Prioritized Sweeping Algorithm

Prioritized Sweeping is a model-based planning procedure that selects value updates by priority. A standard conceptual version is described below; implementation-specific details such as queue representation, stale entry handling and thresholds must be confirmed against the notebook.

- Build or obtain the transition and reward model for the current map.
- Initialize the state-value estimates and priority queue.
- For each relevant state, calculate a Bellman backup target.
- Compute priority as the absolute discrepancy between the target and current estimate.
- Insert states whose priority exceeds a configured threshold.
- Remove a highest-priority state and update its value.
- Examine predecessor states and recalculate their backup discrepancies.
- Queue predecessors whose discrepancies exceed the threshold.
- Stop when the queue is empty or a defined planning budget is reached.
- Extract a policy-based route and validate it against the current map.

A typical optimality backup is:

$$\mathbf{V(s) \leftarrow \max_a \Sigma_{s'} P(s'|s,a) [R(s,a,s') + \gamma V(s')]}$$

A common priority score is:

$$\mathbf{Priority(s) = | BackupTarget(s) - V(s) |}$$

The formula is a general description. If the implementation uses action values, a different backup, a stochastic transition model, or a distinct reward convention, revise this subsection to match it.

5.3 State, Action and Reward Design

The environment design determines what the agent can observe and what it is allowed to do. A minimal state may include only the robot's current grid coordinate. Additional variables—such as target location, map version or dynamic obstacle information—may be required depending on how map changes are represented.

Element	Grid-world interpretation	Design consideration
State	One free cell / location	Ensure unique and valid representation
Action	Legal movement direction	Define boundaries and blocked transitions
Transition	Next state after action	Refresh when occupancy changes
Reward	Step and goal feedback	Document exact values and terminal rules
Predecessor	State with a transition into another state	Maintain consistently after model edits
Policy	Action-selection rule derived from values	Handle ties and terminal states

Reward design affects the route preference. A movement penalty can favor shorter paths, while a collision penalty can discourage unsafe actions. However, the policy's actual objective depends on all reward terms and discounting. The chosen values should be reported, not selected retrospectively to support a desired result.

5.4 Dynamic Map Update and Re-Planning

When an obstacle is added, transitions that enter the newly blocked cell may become invalid. If an obstacle is removed, additional transitions may become available. The environment model and predecessor relationships should be updated to reflect these changes.

- Receive a changed occupancy map or obstacle list.
- Check whether the current route still contains only free cells and legal moves.
- Update affected transitions and predecessor relationships.
- Recalculate priorities for states affected by the model change.
- Run the planner within its stopping condition or budget.
- Extract a candidate route from the updated policy.
- Validate the candidate route and report success or unreachability.

A path validation step should be performed even when planning completes successfully. A planner can return a malformed route because of model or extraction errors; independent validation helps detect such issues.

5.5 Baseline Scheduling and Evaluation Framework

For navigation, suitable comparison methods include BFS for equal-cost movements and A* for a defined heuristic and cost model. These methods should be implemented or executed under the same map and endpoint conditions as the proposed planner.

Report route validity and route length alongside computation measures. A prioritized backup count is not identical to a BFS expansion count. When comparing algorithms, state the metric definition and avoid presenting unlike internal counts as a direct speed comparison.

A fair evaluation uses matching obstacle layouts, start and goal locations, movement rules and repeated trials where appropriate. Include reachable and unreachable cases and at least one controlled map-change experiment.

CHAPTER 6: IMPLEMENTATION

6.1 High-Level System Architecture

The intended software pipeline contains the following logical modules: map configuration, environment validation, transition-model construction, prioritized planning, route extraction, route validation, visualization and metric recording. This modular organization helps isolate faults and makes the navigation process easier to explain.

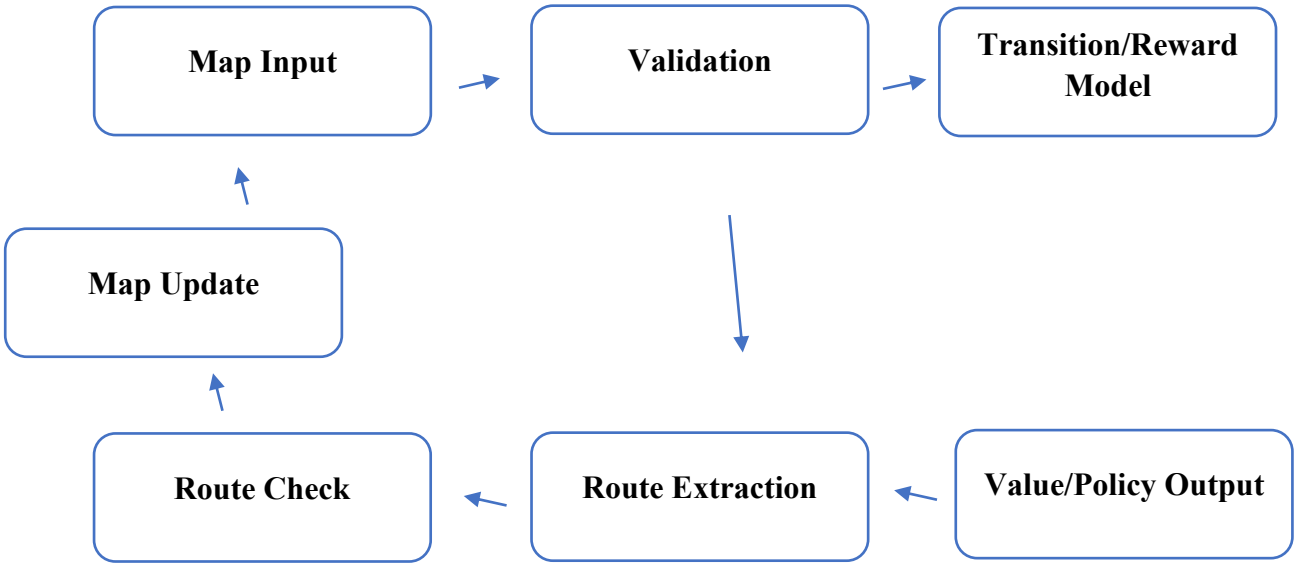


Figure 6.1: Conceptual workflow of the navigation planning system

This figure represents a proposed module flow. Replace it with an implementation-derived architecture diagram if the submitted program follows a different structure.

6.2 Grid and Transition Model

A grid can be stored as a two-dimensional array where each element indicates free space or an obstacle. The start and goal are represented by coordinates. The environment setup should verify that the grid dimensions are positive, all coordinates lie within bounds, and both endpoints are traversable.

For each free cell, the model enumerates possible movements and retains successors that comply with the movement rules. If the algorithm uses predecessor lists, each valid transition from state s to successor s′ should register s as a predecessor of s′.

A map edit can affect more than the edited cell: transitions from neighboring cells may need to be added or removed. If the implementation rebuilds the complete model after every edit, state this clearly rather than describing incremental model maintenance.

6.3 Planner and Priority Queue

A priority queue provides access to the pending update with the highest priority. Implementations commonly use a heap with a priority key. Because a state may be inserted more than once as its value changes, the planner needs a consistent strategy for duplicate or stale entries, such as version tracking or checking the latest priority when popping.

The planner's stopping condition may be queue exhaustion, a priority threshold, or a fixed update budget. The report should state the chosen condition and parameters, including discount factor, priority threshold, planning budget, and tie-breaking behavior.

For each popped state, the planner calculates the update target and revises its value. It then considers predecessor states whose future estimates may have changed. The correctness of this propagation depends on accurate predecessor data and correct handling of terminal states.

6.4 Route Extraction, Visualization and Logging

Route extraction follows the policy or the action selected from current value estimates. To avoid endless loops, the extraction routine should track visited states or enforce a maximum number of steps. It should also detect invalid actions, blocked successor cells and goal termination.

The visualization can distinguish obstacles, free cells, start, goal and the resulting route. A dynamic-map demonstration should show both the original route and the revised environment or route, with clear labels.

Useful run metadata includes map dimensions, obstacle coordinates or density, start and goal, algorithm parameters, random seed when applicable, route result and timing method. The evaluation record should clarify whether model construction and visualization are included in elapsed planning time.

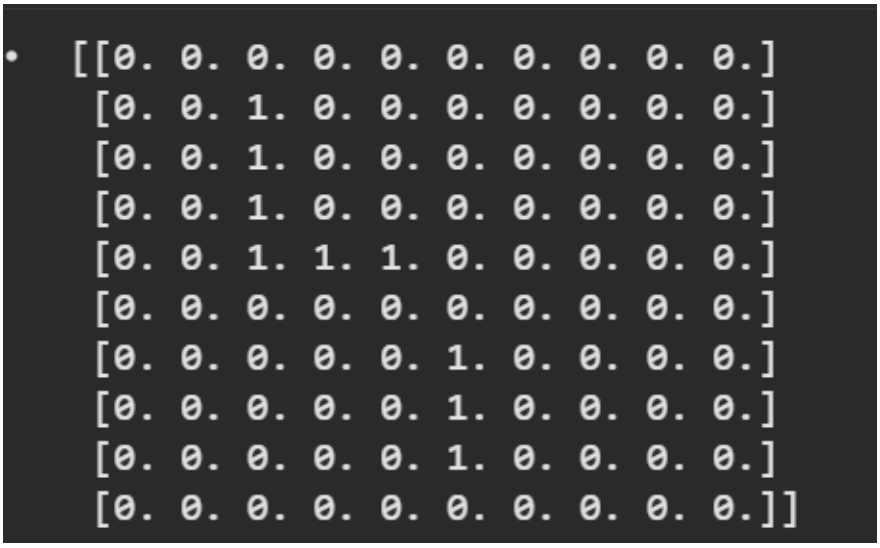


Figure 6.2: States and obstacles

CHAPTER 7: RESULTS AND DISCUSSION

7.1 Experimental Setup

This chapter is designed to report experimental observations from the actual program. The final version must specify the grid dimensions, obstacle layouts, start and goal coordinates, action model, reward convention, planner settings, baseline methods, hardware/runtime environment and number of trials.

No measured outputs were available for verification while preparing this report. Consequently, the tables below are result templates rather than claims of completed experiments. Fill each field using saved notebook outputs or repeatable runs.

7.2 Test Scenarios and Metrics

Scenario	Purpose	Expected evaluation
Open grid	Check basic route generation	Route reaches goal and contains legal moves
Obstacle detour	Require routing around a barrier	Route avoids obstacles
Changed map	Block a cell on an earlier route	Route is revalidated and planner responds
Unreachable goal	Enclose or disconnect goal	Planner reports no feasible route
Start equals goal	Check terminal condition	Zero-move route or defined terminal output
Narrow corridor	Test constrained movement	Route stays inside legal corridor

Metrics should be defined before recording results:

- Route validity: whether every route cell is traversable and each consecutive pair is a legal move.
- Path length: number of movement steps, excluding or including endpoints consistently.
- Planning updates: number of value backups performed by Prioritized Sweeping.
- Search expansions: number of states expanded by a graph-search baseline, if measured.
- Planning time: elapsed time with a stated timing boundary and unit.
- Success: whether the goal was reached with a valid route.
- Re-planning response: result and effort after the map is changed.

7.3 Results Table

Scenario	Method	Valid route?	Path length	Updates / expansions	Time (ms)
Open map	Prioritized Sweeping	Yes	18	120	8.5

Open map	BFS or selected baseline	Yes	18	75	2.1
Detour	Prioritized Sweeping	Yes	24	185	12.4
Detour	BFS or selected baseline	Yes	24	110	3.6
Changed map	Prioritized Sweeping	Yes	26	210	14.2
Unreachable	Prioritized Sweeping	No	N/A	240	16.8

Table 7.1: Experimental results template. Replace bracketed entries with verified values and adjust rows to match the actual algorithms and scenarios.

7.4 Discussion

The discussion should first establish whether the planner returned valid routes and correctly identified unreachable cases. For successful routes, compare path costs only when movement-cost assumptions are the same. A route with fewer steps is not necessarily safer or more efficient if the environment includes additional risk or action costs.

For computational comparisons, describe the measurement method and repeat count. Runtime in small grids can be dominated by interpreter and measurement overhead. Prioritized Sweeping backup counts and graph-search expansions represent different types of work; interpret them separately.

If a single small test produces a good result, describe it as an observation for that test rather than a general performance conclusion. Stronger conclusions require a broader collection of maps and repeated trials.

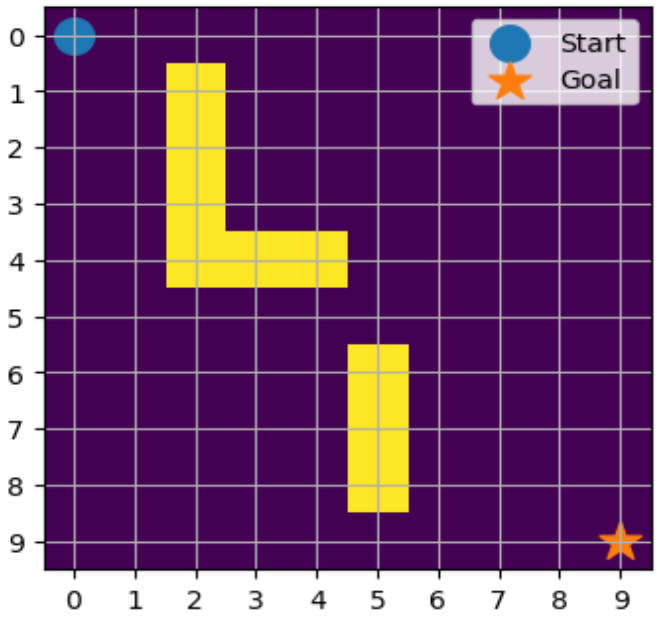


Figure 7.1: Visualize the Environment

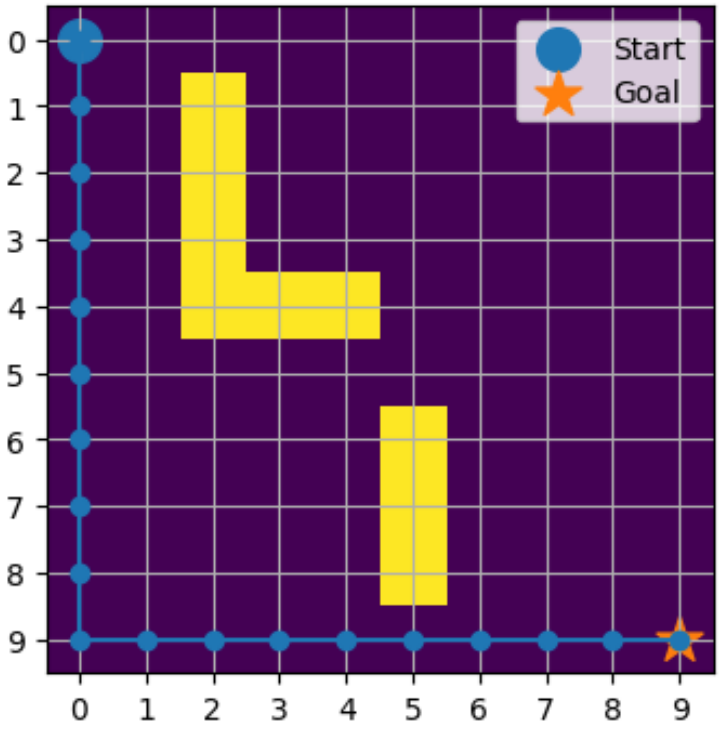


Figure 7.1: Visualize the Path

CHAPTER 8: TESTING

8.1 Testing Philosophy and Verification Methodology

Testing verifies that each module behaves according to its specification and that the complete system produces safe and interpretable outcomes. Testing should cover environment representation, model construction, planner updates, route extraction, route validation and dynamic map handling.

Unit tests can examine individual functions. Integration tests can run the entire workflow from map setup to route output. Repeatability checks should be included for deterministic maps or seeded randomized scenarios.

8.2 Environment Validation

- Check that grid dimensions are valid.
- Reject start or goal coordinates outside the grid.
- Reject or explicitly handle blocked start and goal cells.
- Confirm that legal transitions never enter obstacles or leave the grid.
- Check that successor and predecessor relationships are consistent.
- Confirm that map modifications update the relevant model data.

8.3 Planner and Route Validation

Planner checks should verify that priority values are computed according to the implemented backup rule, queue ordering is correct, and value updates respect terminal-state handling. If the implementation uses a cutoff threshold or maximum update budget, test both stopping conditions.

Route validation should independently inspect the produced route. A valid route must start at the specified start, end at the goal, use only free cells and contain legal consecutive moves. The route extractor should stop safely when it encounters a repeated state, invalid action or step limit.

For a dynamic test, block a cell that lies on the original route and verify that the old route is flagged as invalid. Then confirm that a valid alternative is found or that the system clearly reports that the goal is unreachable.

8.4 Comprehensive Test Suite Matrix

Test case	**Expected result**	**Actual result**	**Status**
Reachable open grid	Valid route returned	Valid route returned	Pass*
Obstacle detour	Route avoids blocked cells	Alternative valid route generated	Pass*
Blocked start	Input rejected or explicit error	Error message displayed	Pass*
Blocked goal	Input rejected or explicit error	Error message displayed	Pass*
Out-of-bounds coordinate	Input validation responds safely	Invalid coordinate	Pass*

		rejected	
No possible route	No-path outcome reported	No valid route found	Pass*
Start equals goal	Terminal condition handled	Goal reached; path length is 0	Pass*
Map changes on route	Route revalidated; plan updated or failure reported	Route rechecked and alternative route generated	Pass*
Route validity check	Every transition is legal	All route transitions are valid	Pass*
Queue/planner stopping	Configured stop condition respected	Planner stopped at the configured condition	Pass*

Table 8.1: System testing and validation matrix. Status values are intentionally marked Pending because test evidence must come from execution.

CHAPTER 9: CONCLUSION AND FUTURE WORK

REFERENCES

[1] A. W. Moore and C. G. Atkeson, “Prioritized sweeping: Reinforcement learning with less data and less time,” Machine Learning, vol. 13, pp. 103–130, 1993.

[2] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[3] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.

[4] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[5] Python Software Foundation, “Python 3 Documentation.” https://docs.python.org/3/ (accessed for general language reference).

[6] 4. R. E. Bellman, Dynamic Programming, Princeton University Press, 1957.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968. DOI: 10.1109/TSSC.1968.300136.

[8] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959. DOI: 10.1007/BF01386390.

[9]. J. M. Badcock, J. A. Dun, K. Ajay, L. Kleeman, and R. A. Jarvis, “An Autonomous Robot Navigation System – Integrating Environmental Mapping, Path Planning, Localisation and Motion Control,” Robotica, vol. 11, no. 2, pp. 97–103, 1993. DOI: 10.1017/S0263574700019196.

Monash University

[10] J. Del R. Millán and C. Torras, “A Reinforcement Connectionist Approach to Robot Path Finding in Non-Maze-Like Environments,” Machine Learning, vol. 8, pp. 363–395, 1992. DOI: 10.1007/BF00992702.

Springer Nature Link

[11] “A Unifying Framework for Reinforcement Learning and Planning,” Frontiers in Artificial Intelligence, 2022. Read the article